Project Report Format

1. INTRODUCTION
- 1.1 Project Overview
- 1.2 Purpose

2. IDEATION PHASE
- 2.1 Problem Statement
- 2.2 Empathy Map Canvas
- 2.3 Brainstorming

3. REQUIREMENT ANALYSIS
- 3.1 Customer Journey map
- 3.2 Solution Requirement
- 3.3 Data Flow Diagram
- 3.4 Technology Stack

4. PROJECT DESIGN
- 4.1 Problem Solution Fit
- 4.2 Proposed Solution
- 4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING
- 5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING
- 6.1 Performance Testing

7. RESULTS
- 7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX
- Source Code(if any)
- Dataset Link
- GitHub & Project Demo Link

INTRODUCTION:

1.1 Project Overview:

The Smart Library Request Workflow in ServiceNow is designed to automate and manage library operations efficiently. The system defines roles such as students and librarians, configures custom tables (Book and Borrow Request), implements workflow automation for approvals, enforces field and access control, and generates reports to monitor library activities. The backend setup ensures role-based access, proper tracking of issued books, and timely notifications for requests.

1.2 Purpose:

The purpose of this project is to simplify library management by:

- Automating book borrowing and approval processes.
- Restricting access to data based on roles.
- Preventing conflicts like double-booking of issued books.
- Providing actionable insights to administrators through reports.

IDEATION PHASE:

2.1 Problem Statement:

Manual library operations can lead to errors in book issuance, delayed approvals, and difficulty tracking borrowed books. Students may request unavailable books, and librarians may spend significant time managing approvals and records.

2.2 Empathy Map Canvas:

- **Users:** Students, Librarians
- **Needs:** Quick access to book requests, approval automation, status visibility.
- **Pains:** Manual tracking of borrowed books, multiple approvals, mismanagement.
- **Gains:** Faster approvals, reduced manual effort, accurate book availability tracking.

2.3 Brainstorming:

Ideas considered:

- Manual vs automated request approval
- Adding notifications for requests
- Preventing already issued books from being requested
- Generating reports for library usage analytics

REQUIREMENT ANALYSIS:

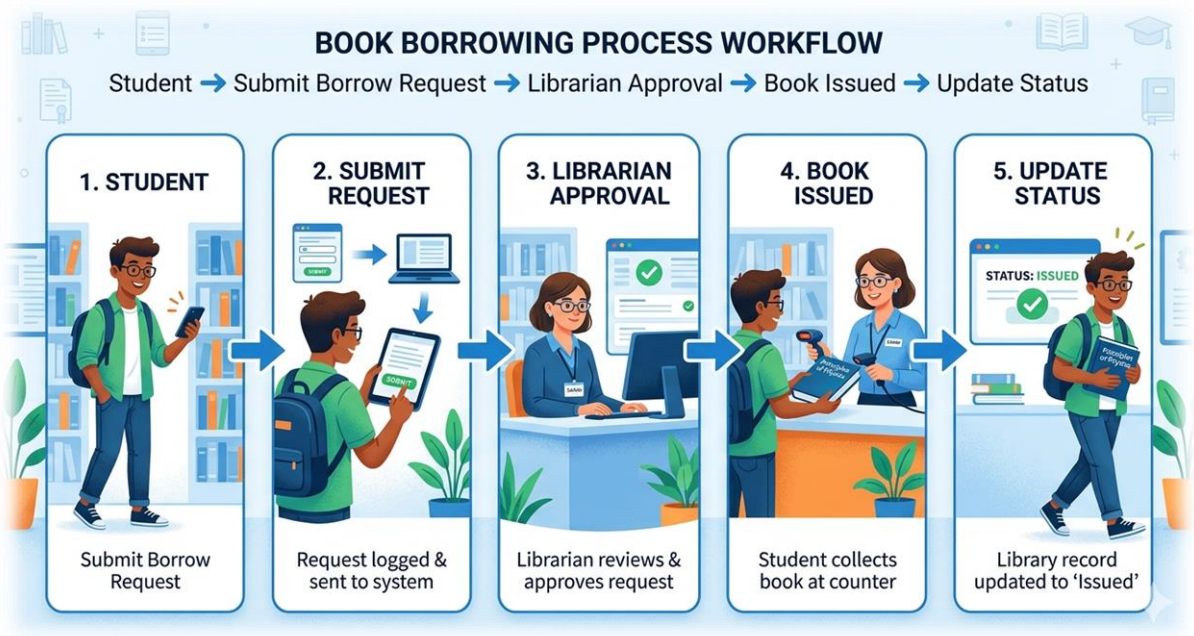
3.1 Customer Journey map

- **Students:** Browse catalog → Request Book → Receive Approval → Collect Book → Return Book
- **Librarians:** Receive Requests → Approve/Reject → Update Book Status → Monitor Reports

3.2 Solution Requirement

- **Roles:** Student, Librarian
- **Tables:** Book, Borrow Request
- **Automation:** Flow Designer to approve and issue books
- **UI Policies:** Lock fields after approval
- **ACLs:** Role-based access control

3.3 Data Flow Diagram



3.4 Technology Stack

- **Platform:** ServiceNow
- **Components:** Flow Designer, UI Policies, ACLs, Reports
- **Database:** Custom Tables (Book & Borrow Request)
- **Users & Roles:** Student, Librarian

PROJECT DESIGN:

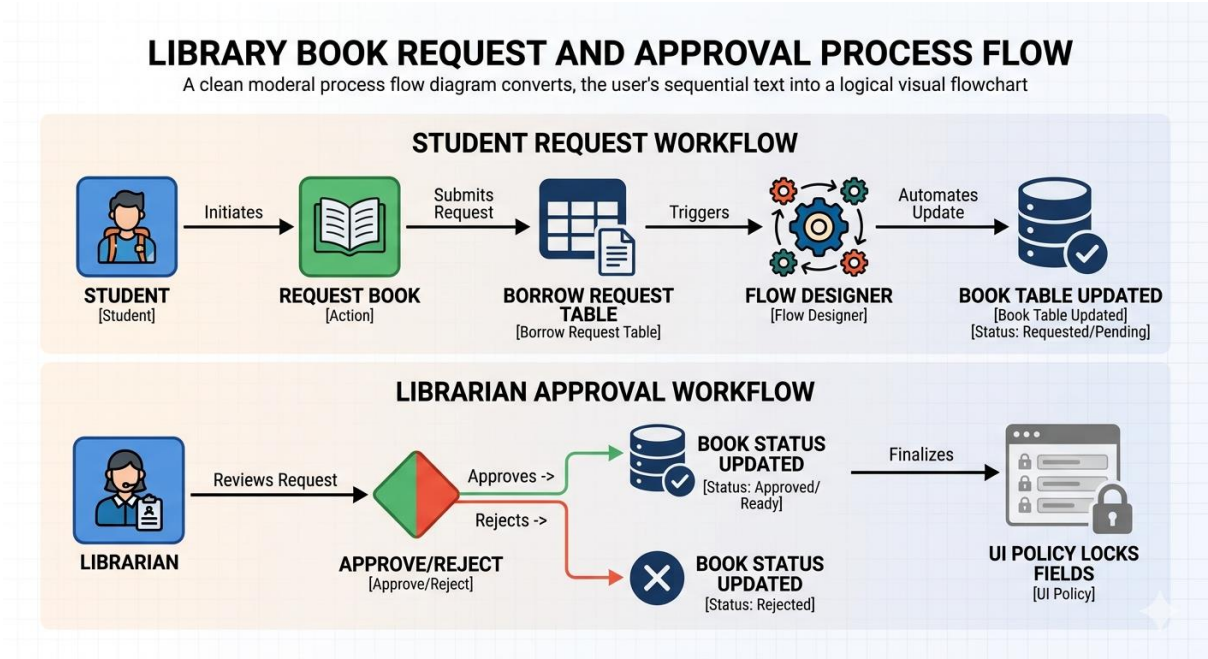
4.1 Problem Solution Fit

The solution automates book issuance, prevents double-booking, and provides clear access control for students and librarians. Reports improve library management and monitoring.

4.2 Proposed Solution

- Create roles: Student and Librarian
- Configure Book & Borrow Request tables with necessary fields
- Implement Flow Designer automation for request approval → issue book
- Lock fields after approval using UI Policies
- Implement ACLs to restrict data access based on roles
- Generate "Most Borrowed Books" report for administrators

4.3 Solution Architecture



PROJECT PLANNING & SCHEDULING:

5.1 Project Planning

Milestone	Task	Timeline
1	Create Roles (Student, Librarian)	Day 1
2	Create Tables (Book, Borrow Request)	Day 2
3	Add Fields to Tables	Day 3
4	Configure Related Lists & Reference Qualifiers	Day 4
5	Implement Flow Designer Automation	Day 5
6	Apply UI Policies	Day 6
7	Define ACLs	Day 7
8	Build Reports	Day 8

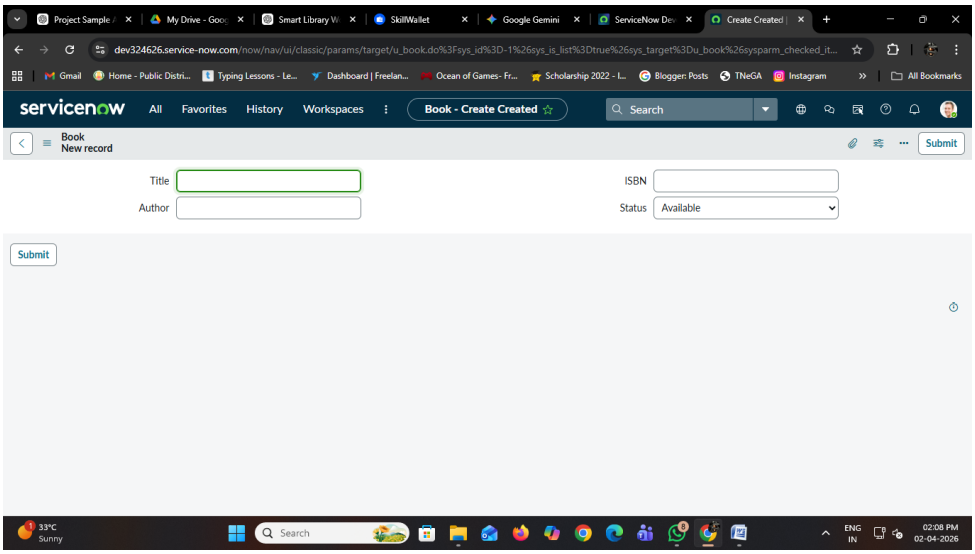
FUNCTIONAL AND PERFORMANCE TESTING:

6.1 Performance Testing

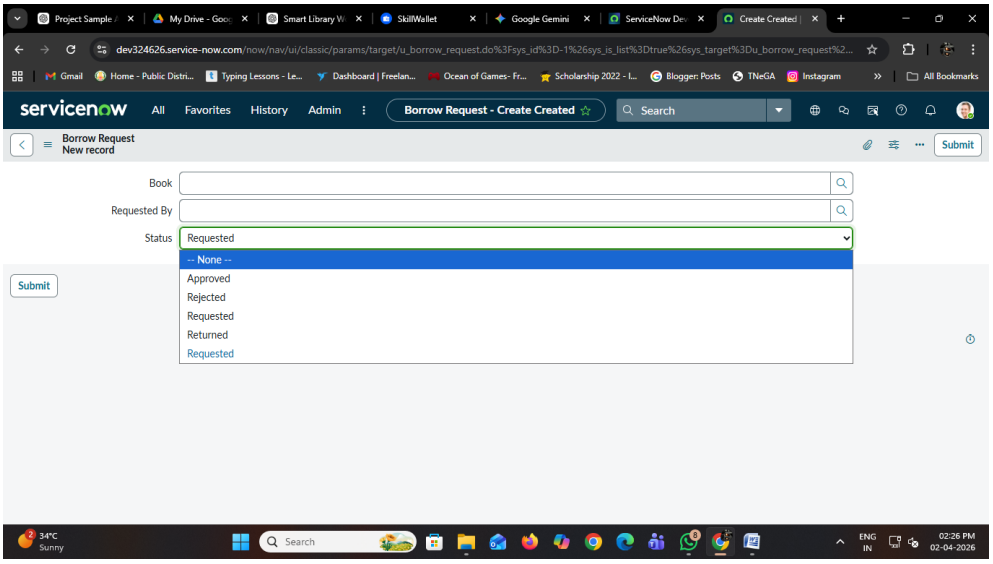
Test Case ID	Scenario	Steps	Expected Result	Actual Result	Pass/Fail
FT-01	Request Book	Student requests available book	Request created successfully	Passed	Pass
FT-02	Approve Request	Librarian approves request	Book status updates to Issued	Passed	Pass
FT-03	Prevent Double Issue	Student requests already issued book	Book not listed	Passed	Pass

RESULTS:

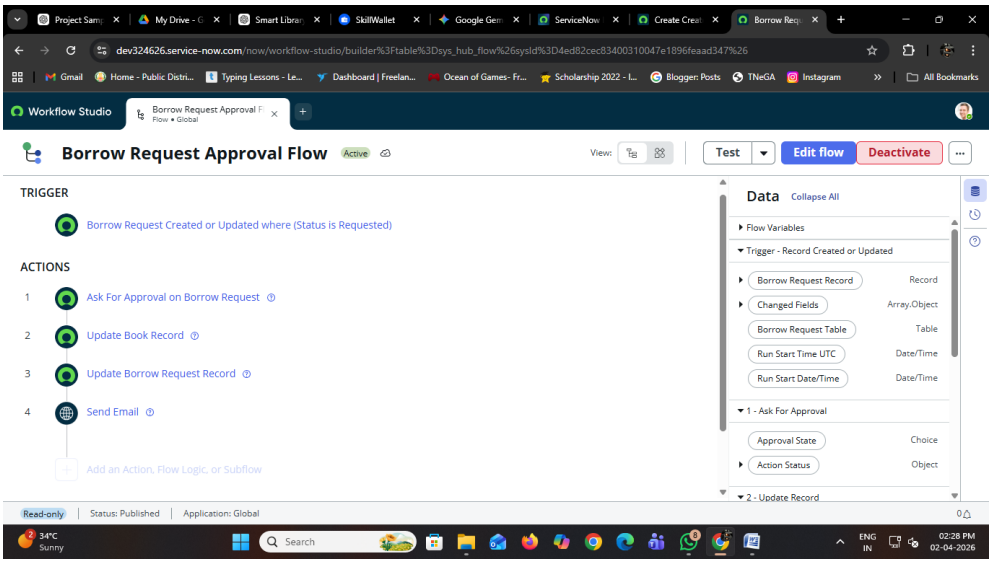
- Screenshot of Book table with status



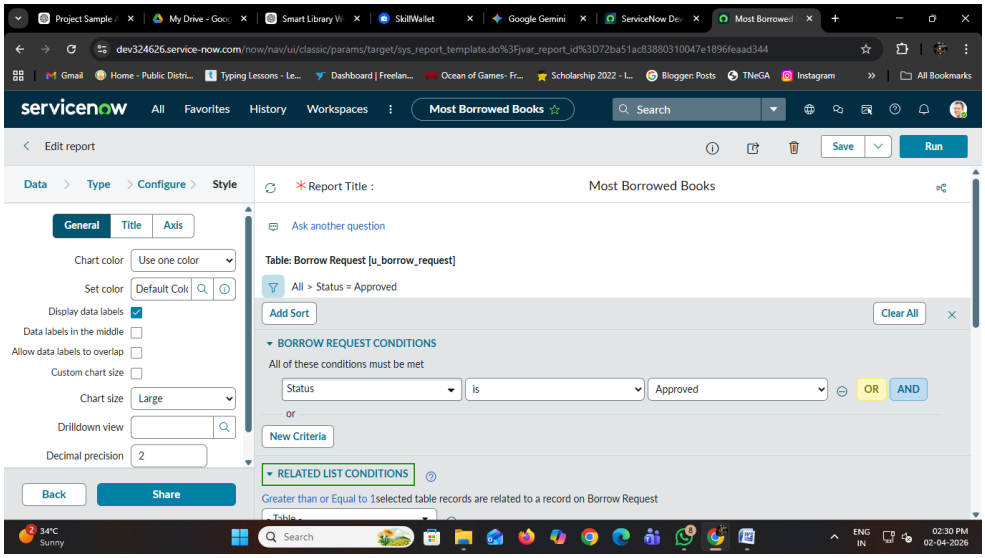
- Borrow Request form before and after approval



- Flow Designer configuration



- “Most Borrowed Books” report



ADVANTAGES & DISADVANTAGES:

Advantages:

- Automated approval workflow
- Accurate tracking of borrowed books
- Role-based access control
- Insights via reports

Disadvantages:

- Requires ServiceNow subscription
- Initial setup can be complex

CONCLUSION:

The Smart Library Request Workflow in ServiceNow provides a robust, automated solution for managing library operations efficiently. It minimizes manual errors, improves tracking, and enhances user experience for both students and librarians.

FUTURE SCOPE:

- Integration with RFID for automatic book issue/return
- SMS/WhatsApp notifications for requests
- Analytics dashboard for real-time monitoring
- Mobile-friendly UI for students

APPENDIX:

Source Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Smart Library Workflow</title>

<style>
  body {
    font-family: Arial;
    margin: 20px;
    background-color: #f4f6f9;
  }
  .container {
    max-width: 1000px;
    margin: auto;
    background: white;
    padding: 20px;
    border-radius: 10px;
  }
  table {
    width: 100%;
    border-collapse: collapse;
  }
  table, th, td {
    border: 1px solid #ccc;
  }
  th, td {
    padding: 10px;
  }
  button {
    padding: 6px 10px;
    margin: 3px;
    cursor: pointer;
    background: #3498db;
    color: white;
    border: none;
    border-radius: 5px;
  }

  .available { color: green; font-weight: bold; }
  .issued { color: orange; font-weight: bold; }
```

```
    .returned { color: blue; font-weight: bold; }
    .lost { color: red; font-weight: bold; }

</style>
</head>

<body>
<div class="container">

<h1>□ Smart Library Workflow</h1>

<!-- Book Table -->
<h2>□ Books</h2>
<table id="bookTable"></table>

<!-- Request -->
<h2>□ Borrow Request</h2>
<input type="text" id="user" placeholder="Enter student name">
<select id="bookSelect"></select>
<button onclick="requestBook()">Request</button>

<!-- Requests -->
<h2>□ Requests</h2>
<table id="requestTable"></table>

<!-- Report -->
<h2>□ Most Borrowed Books</h2>
<ul id="report"></ul>

</div>

<script>
let books = [
    { title: "Introduction to Java Programming", author: "Herbert
Schildt", isbn: "9780071809252", status: "Available", count: 5 },
    { title: "Data Structures and Algorithmss", author: "Thomas H.
Cormen", isbn: "9780262033848", status: "Available", count: 8 },
    { title: "Computer Networks", author: "Andrew S. Tanenbaum",
isbn: "9780132126953", status: "Available", count: 6 },
    { title: "Cyber Security Essentials", author: " Charles J. Brooks",
isbn: "9781119362395", status: "Available", count: 7 },
    { title: "Artificial Intelligence Basics", author: "Tom Taulli",
isbn: "9781484250273", status: "Available", count: 9 }
```

```
];

let requests = [];

// Load Books
function loadBooks() {
    let table = document.getElementById("bookTable");

    table.innerHTML = `
    <tr>
        <th>Title</th>
        <th>Author</th>
        <th>ISBN</th>
        <th>Status</th>
    </tr>`;

    let select = document.getElementById("bookSelect");
    select.innerHTML = "";

    books.forEach((b, i) => {

        let statusClass = b.status.toLowerCase();

        table.innerHTML += `
        <tr>
            <td>${b.title}</td>
            <td>${b.author}</td>
            <td>${b.isbn}</td>
            <td class="${statusClass}">${b.status}</td>
        </tr>`;

        if (b.status === "Available") {
            select.innerHTML += `<option
value="${i}">${b.title}</option>`;
        }
    });
}

// Request Book
function requestBook() {
    let user = document.getElementById("user").value;
    let bookIndex = document.getElementById("bookSelect").value;
```

```
    if (!user) {
        alert("Enter name");
        return;
    }

    requests.push({
        user: user,
        bookIndex: bookIndex,
        status: "Requested"
    });

    loadRequests();
}

// Load Requests
function loadRequests() {
    let table = document.getElementById("requestTable");

    table.innerHTML = `
    <tr>
        <th>User</th>
        <th>Book</th>
        <th>Status</th>
        <th>Action</th>
    </tr>`;

    requests.forEach((r, i) => {
        let actions = "";

        if (r.status === "Requested") {
            actions = `
                <button onclick="approve(${i})">Approve</button>
                <button onclick="reject(${i})">Reject</button>
            `;
        }

        if (r.status === "Approved") {
            actions = `<button onclick="returnBook(${i})">Return
Book</button>`;
        }

        table.innerHTML += `
        <tr>
```

```
            <td>${r.user}</td>
            <td>${books[r.bookIndex].title}</td>
            <td>${r.status}</td>
            <td>${actions}</td>
        </tr>`;
    });

    generateReport();
}

// Approve
function approve(i) {
    requests[i].status = "Approved";

    let book = books[requests[i].bookIndex];
    book.status = "Issued";
    book.count++;

    loadBooks();
    loadRequests();
}

// Reject
function reject(i) {
    requests[i].status = "Rejected";
    loadRequests();
}

// Return Book
function returnBook(i) {
    requests[i].status = "Returned";

    let book = books[requests[i].bookIndex];
    book.status = "Available";

    loadBooks();
    loadRequests();
}

// Report
function generateReport() {
    let report = document.getElementById("report");
    report.innerHTML = "";
```

```
    let sorted = [...books].sort((a,b)=>b.count-a.count);

    sorted.slice(0,5).forEach(b=>{
        report.innerHTML += `<li>${b.title} - ${b.count}
times</li>`;
    });
}

// Init
loadBooks();
</script>

</body>
</html>
```

GitHub & Project Demo Link:

1. GitHub Link: https://github.com/kalaisivakumar13-sudo/Smart-Library-Request-Workflow-in-ServiceNow.git

2. Project Demo Link: https://drive.google.com/drive/folders/11TfQdluQ-6ZhUIIDX7y4pP9dPvWy18KO?usp=drive_link